ENGINEERING HANDOFF / ITERATION 1

HALITE

FYP-I MID

THREE-MEMBER EXECUTION BLUEPRINT

Maintenance Diagnostics, Knowledge Graph, Conflict Intelligence, CI/CD

One shared reference for the team and AI coding agents

Shahmeer Atif	23I-0711 – Frontend / UX / API integration
Muhammad Umar	23I-0782 – Graph / diagnostic retrieval / ranking
Muhammad Usman Al Haq	23I-0585 – Data / conflict / testing / DevOps

Supervisor: Syed Muhammad Saad Salman

FAST School of Computing, NUCES Islamabad

Mid-I announced window: 12–16 October 2026

Version 1.0 – 15 October 2026 – Working implementation contract

Read This First

This is a work order, not an implementation-complete report. It specifies what to audit, preserve, modify, test and deliver. Do not mark tasks complete simply because an AI agent says it wrote the code. Completion requires an executed test, a working integration, and independently reviewed evidence.

Deadline rule. The FYP Committee's message sets Week 9 at **12–16 October 2026**. A separately mentioned 19 October date is *not* treated as an automatic extension. Confirm a different date with the supervisor in writing. Target a frozen, demonstrated version by 16 October if possible.

Repository rule. The supplied archive `Halite(FYP).zip` includes a large `.git` directory, a Windows virtual environment, installed packages, an existing React/Vite frontend, a FastAPI backend, Neo4j-related services, and tests. *Never* overwrite the working project with a scaffold generated from scratch. Work from the actual repository and inspect its current branch before making changes.

Scope rule. Maintenance diagnostics is the focus. Software-domain features and MCP work are dropped from the *product scope*, but their legacy files should be isolated or disabled safely; do not delete broad shared infrastructure without tests.

Source of truth hierarchy: (1) working source and executed tests; (2) Mid-I assessment instructions; (3) approved project scope and source-derived documentation; (4) this execution plan; (5) agent-generated suggestions. When these conflict, record the discrepancy and obtain team approval.

Confidence statements. Existing POC benchmarks concern conflict detection, not proven diagnostic accuracy. A case-based exploratory diagnostic ranker is already present. Do not promise an experimentally validated diagnosis model at Mid-I.

Contents

Read This First i

1 Mission, Scope and Acceptance Gate 1

1.1 What the Team Must Show . . . 1
1.2 Mandatory Product Workflow . . . 1
1.3 Non-Goals for Mid-I . . . 1
1.4 Exit Definition (All Required) . . . 2

2 What Already Exists: Verified Archive Inventory 3

2.1 Repository Technology and Structure . . . 3
2.2 Existing Maintenance Paths: Do Not Duplicate . . . 3
2.3 Existing Routes and Response Reality . . . 4
2.4 Critical Discovery: Real Neo4j vs Demo Mode . . . 4
2.5 Pre-existing Technical Risks to Audit . . . 5

3 Team Ownership and Zero-Overlap Rules 6

3.1 Ownership Principle . . . 6
3.2 Shared Contracts: Controlled Owner . . . 6
3.3 Protected Files, Especially During Mid-I . . . 7
3.4 Branch Names . . . 7

4 Exact Interface Contract Freeze 8

4.1 Freeze Window and Contract Changes . . . 8
4.2 Existing Diagnose Request . . . 8
4.3 Existing Diagnose Response: Field Expectations . . . 8
4.4 Other Existing Endpoints . . . 8
4.5 Error Response Agreement . . . 9
4.6 Cross-Agent Integration Handshake . . . 9

5 Shahmeer Atif: Frontend and Diagnostic Experience 10

5.1 Domain and Exclusions . . . 10
5.2 Work Package S1: POC Preservation and Cleanup . . . 10
5.3 Work Package S2: Asset List and History . . . 10
5.4 Work Package S3: Diagnostic Workspace . . . 11
5.5 Work Package S4: Evidence Navigation and Review UI . . . 11

5.6 Work Package S5: Frontend Quality and Tests . . . 11
5.7 Shahmeer: Strict Do-Not-Touch List . . . 12

6 Muhammad Umar: Graph, Retrieval and Ranking 13
6.1 Domain and Exclusions . . . 13
6.2 Work Package U1: Audit Existing Diagnostic Implementation . . . 13
6.3 Work Package U2: Real-Data Graph Retrieval . . . 13
6.4 Work Package U3: Deterministic Initial Fault Ranking . . . 13
6.5 Work Package U4: Citation Validation and LLM Boundary . . . 14
6.6 Work Package U5: Diagnostic Backend Tests . . . 14
6.7 Umar: Explicitly Deferred Until After Mid-I . . . 14

7 Muhammad Usman Al Haq: Data, Conflict and Delivery 15
7.1 Domain and Exclusions . . . 15
7.2 Work Package A1: Data and Environment Baseline . . . 15
7.3 Work Package A2: Maintenance APIs and Conflict Workflow . . . 15
7.4 Work Package A3: GitHub Actions and Repository Health . . . 15
7.5 Work Package A4: Backend Wiring and Documentation . . . 16
7.6 Usman: Special Data and Threshold Caveats . . . 16
7.7 Usman: Deliverable Handoff . . . 16

8 Day-by-Day Integration Protocol 17
8.1 Day 0: Contract Lock (Immediately) . . . 17
8.2 Day 1: Parallel Development . . . 17
8.3 Day 2: Integrate Through Public Interfaces . . . 17
8.4 Day 3: Release and Rehearsal . . . 17
8.5 Late Deadline Condition . . . 18

9 Git and AI Agent Collaboration Rules 19
9.1 Safe Branch Routine . . . 19
9.2 What Every Agent Must Do Before Coding . . . 19
9.3 What Every Agent Must Never Do . . . 19
9.4 PR Template (Use as Written) . . . 20
9.5 Conflict Resolution Policy . . . 20

10 University Mid-I Deliverables and Evidence Map 21
10.1 Rubric-Aligned Evidence . . . 21
10.2 Report Assembly Guidance . . . 22
10.3 Evidence Folder Layout (New, Proposed) . . . 22

11 Test Cases, Failure Cases and Acceptance Matrix **23**
11.1 No-Mocking Rule for the Evaluator Demo . . . 24
11.2 Post-Integration Smoke Commands . . . 24

12 Live Mid-I Demonstration Script and Defense Preparation **25**
12.1 Demo Order (Suggested 8–12 Minutes) . . . 25
12.2 Questions Shahmeer Must Answer . . . 25
12.3 Questions Umar Must Answer . . . 25
12.4 Questions Usman Must Answer . . . 26
12.5 Adversarial Demo Cases . . . 26

13 Risk Register and Release Freeze **27**
13.1 Release Candidate Check . . . 27

14 Ready-to-Paste Instructions for Coding Agents **29**
14.1 Global Instructions (Give to All Three Agents) . . . 29
14.2 Shahmeer Agent Prompt . . . 29
14.3 Umar Agent Prompt . . . 29
14.4 Usman Agent Prompt . . . 30
14.5 Agent Stop-and-Ask Conditions . . . 30

A Mid-I Team Final Checklist **31**
A.1 Shahmeer . . . 31
A.2 Umar . . . 31
A.3 Usman . . . 31
A.4 Whole Team . . . 31

B Evidence Log Template **32**

C Source References and Decision Notes **33**

1 Mission, Scope and Acceptance Gate

1.1 What the Team Must Show

The deliverable is **one working vertical slice**, not three disconnected demos. A technician chooses a real maintenance asset; inspects dated records; enters a symptom; receives a graph-grounded, evidence-linked diagnostic investigation response; can open source evidence; and sees at least one real historical correction or conflict example. The same commit must have test evidence and a meaningful GitHub CI workflow.

1.2 Mandatory Product Workflow

1. Load Halite's maintenance-only UI; identify the demo dataset and runtime mode.
2. Open Assets; choose a known asset from the *actual loaded corpus*.
3. Inspect its dated ServiceRecord timeline and original record identifiers.
4. Open Diagnostics with the selected asset; enter a descriptive symptom.
5. Send a real POST request to the existing diagnosis endpoint; show loading, error and success states.
6. Show ranked investigative checks, supporting record identifiers, explanation and citation-validation status.
7. Open at least one cited record in the real source/history interface.
8. Open a real conflict/review example and explain whether it is an allegation, a correction, or an adjudicated relationship.
9. Show GitHub PR/branch evidence and automatic tests; demonstrate healthy running services.
10. Demonstrate a failure case (unknown asset, no data, missing live LLM, or model rate limit) without misleading generated output.

1.3 Non-Goals for Mid-I

- No new PostgreSQL/SaaS multi-tenancy migration at this stage unless already implemented.
- No complete Technician Academy, predictive maintenance model, inventory forecasting, or large-scale model training.
- No full UI redesign that delays integration or changes existing API contracts without agreement.
- No reimplementation of a functioning conflict engine simply to use a different library.
- No silent switch to in-memory demo data or hardcoded AI output when claiming real-data functionality.

1.4 Exit Definition (All Required)

MID-I PASS GATE
A. A functional app with real asset and graph history.
B. Diagnosis API succeeds with the verified live-model path or reports a clearly explained limitation.
C. At least one valid source evidence link opens a real record.
D. Existing conflict workflow is visible and testable.
E. Automated backend tests and frontend tests/build run in GitHub Actions.
F. Repository tag/commit, setup instructions, tests, evidence and owner contributions are documented.
G. Every member can defend their own code and explain an integration failure case.

2 What Already Exists: Verified Archive Inventory

2.1 Repository Technology and Structure

Inspection of the supplied ZIP found the following actual project components. This is file existence and code-structure evidence, *not* proof that every endpoint currently passes integration tests or is deployed.

- **Frontend:** React 18, Vite 5, JavaScript/JSX, Tailwind, Axios, Vitest and Testing Library.
- **Backend:** Python FastAPI, Pydantic, graph repository, Groq wrapper, maintenance services and PyTest tests.
- **Graph:** Neo4j graph client, graph queries and an in-memory fallback mode.
- **Operational assets:** Docker Compose, backend Dockerfile, environment example files, README, scripts and demo documentation.

2.2 Existing Maintenance Paths: Do Not Duplicate

Table 2.1: Source File Inventory and Role

Existing path	Current role / instruction
`frontend/src/App.jsx`	App routing includes both legacy software screens and maintenance screens; Shahmeer adapts navigation, carefully.
`frontend/src/pages/Diagnostics.jsx`	Existing diagnostics screen; Shahmeer enhances rather than replaces.
`frontend/src/pages/Assets.jsx`	Asset search/list; Shahmeer verifies it works with real corpus.
`frontend/src/pages/AssetHistory.jsx`	Asset history; Shahmeer makes record navigation dependable.
`frontend/src/pages/ServiceRecords.jsx`	Record search/view; Shahmeer uses it for cited evidence.
`frontend/src/pages/ReviewQueue.jsx`	Existing reviewer UI; Shahmeer owns UI, Usman owns API.
`frontend/src/components/Sidebar.jsx`	Navigation; Shahmeer removes obsolete visible domains.
`frontend/src/api/client.js`	Existing Axios helpers for maintenance diagnosis, assets, conflict review; Shahmeer owns frontend API changes.
`frontend/src/test/Diagnostics.test.jsx`	Existing Vitest diagnostic tests; Shahmeer extends them.

Existing path	Current role / instruction
`backend/main.py`	FastAPI app and router wiring, health endpoint; Usman is integration owner.
`backend/api/routes/maintenance.py`	Asset, record, ingest and review APIs; Usman owns.
`backend/api/routes/diagnostics.py`	Diagnose and maintenance chat routes; Umar owns.
`backend/services/diagnostic_retrieval.py`	Current graph-grounded historical retrieval; Umar owns.
`backend/graph/queries.py`	Neo4j graph queries; Umar owns.
`backend/graph/schema.py`	Graph schema; Umar owns.
`backend/services/conflict_engine.py`	Conflict candidate selection, classification and routing; Usman owns.
`backend/agents/maintenance_agent.py`	Maintenance ingestion/processing service; Usman owns.
`backend/tests/test_diagnostic_retrieval.py`	Existing backend diagnostic tests; Umar extends.
`backend/tests/test_api_maintenance.py`	Existing maintenance API tests; Usman extends.
`backend/tests/test_conflict_engine.py`	Existing conflict tests; Usman maintains.
`docker-compose.yml`	Local Neo4j/backend services; Usman owns deployment file.
`README.md`	Old README still markets software-team decisions; Usman updates setup text with member review.

2.3 Existing Routes and Response Reality

The backend declares `POST/api/maintenance/diagnose`, `POST/api/maintenance/chat`, `GET/api/maintenance/assets`, `GET/api/maintenance/assets/{asset_id}/history`, `GET/api/maintenance/records/{record_id}`, and review-queue endpoints. The frontend helper layer already calls these endpoints. In the current code, `/diagnose` takes `asset_id` and `symptom_text`; its result contains `ranked_checks`, `evidence`, `rare_cases`, `explanation`, `context_size`, `citation_validation`, and `warnings`.

Do not assume: that session persistence, tenant authentication, full semantic-vector retrieval, or production-ready multi-tenant SaaS is already implemented. These are later-phase commitments in the master report. Current code must be characterized accurately.

2.4 Critical Discovery: Real Neo4j vs Demo Mode

`backend/main.py` can fall back to an in-memory graph, and `/api/health` reports `graph_mode`, `using_memory_graph`, `maintenance_corpus_loaded`, `maintenance_record_count`, and model

availability. **A healthy HTTP status does not establish a healthy real maintenance demonstration.** The gate requires the right runtime mode, corpus presence and functioning record queries.

The current diagnostic route normally raises HTTP 503 when the live model is not available and there is no applicable cached result. The team must demonstrate or disclose this dependency, preserve controlled failure behavior, and avoid presenting mock text as a live diagnosis.

2.5 Pre-existing Technical Risks to Audit

1. **Legacy UI:** App routes, Sidebar and README still reference Chat, Decisions, Gitea and other dropped software-oriented product features. Hide or safely retire their navigation, preserving shared dependencies until tested.
2. **No GitHub Actions workflow observed in archived source inventory:** create an actual `.github/workflows/ci.yml` and verify a workflow run on GitHub. Local tests alone are not CI/CD.
3. **Environmental secrets:** supplied ZIP includes `backend/.env`; do not commit, copy into agent prompts, display in screenshots, or expose real credentials. Audit Git tracking and rotate any credential that was exposed.
4. **Citation validator:** current code verifies record ID existence and flags unmatched IDs. That does not automatically prove every natural-language statement is supported; be precise in claims.
5. **Evaluation threshold:** historical materials describe a very small tuning sample and confidence-threshold bottleneck. Do not advertise a newly validated confidence model at Mid-I.
6. **Data counts:** source documents contain differing corpus summaries; use the count returned by the runtime and label POC-reported figures separately.

3 Team Ownership and Zero-Overlap Rules

3.1 Ownership Principle

One file = one primary editor per sprint. An agent may read any file, but may write only owned paths without a brief, logged handoff. Integration is done via agreed public API contracts, not by crossing into another member's modules.

Table 3.1: Exclusive Edit Responsibility

Member	Edit ownership	Not allowed without handoff
Shahmeer	All `frontend/src/` pages, components, hooks, CSS and `frontend/src/api/client.js`; frontend tests.	No editing `backend/`, Docker, Python data/graph services, CI configuration.
Umar	`backend/api/routes/diagnostics.py`; `backend/services/diagnostic_retrieval.py`; `backend/graph/`; diagnostic backend tests.	No editing frontend, `backend/main.py`, conflict engine, ingest/maintenance routes or workflow YAML.
Usman	`backend/api/routes/maintenance.py`; `backend/services/conflict_engine.py`; `backend/agents/maintenance_agent.py`; `backend/main.py`; Docker/Compose; workflows; setup docs; maintenance/conflict tests.	No editing the diagnostic route or graph implementation without Umar's explicit request; no frontend changes without Shahmeer's approval.

3.2 Shared Contracts: Controlled Owner

- **Backend wire format owner: Umar** for diagnosis, **Usman** for maintenance/review. Shahmeer consumes the response contract and requests changes through a written issue.
- **Frontend Axios adapter owner: Shahmeer**. Only Shahmeer edits the shared `frontend/src/api/client.js`.
- **Backend application/router registration owner: Usman**. Other backend contributors make a short integration request rather than editing `backend/main.py`.
- **Graph schema owner: Umar**. Usman proposes required ingestion fields in an issue; Umar approves schema changes.
- **CI scripts owner: Usman**; each engineer owns the tests for their feature. A test failure is resolved by the feature owner.

- **Report source coordinator: Shahmeer**. Each member contributes evidence through separate files or agreed report sections, not simultaneous editing of the same document.

3.3 Protected Files, Especially During Mid-I

Avoid simultaneous changes to `frontend/src/App.jsx`, `frontend/src/components/Sidebar.jsx`, `frontend/src/api/client.js`, `backend/main.py`, `backend/graph/queries.py`, and `docker-compose.yml`. These are integration hotspots. Their respective owners merge changes deliberately.

3.4 Branch Names

```
main                          # stable demonstration release
develop                       # optional integration branch
feat/mid1/frontend-diagnostics
feat/mid1/graph-diagnostics
feat/mid1/maintenance-devops
fix/mid1/<brief-description>  # only after an owner is assigned
```

No AI agent may independently create additional long-lived integration branches or merge into `main` without the team release coordinator's approval.

4 Exact Interface Contract Freeze

4.1 Freeze Window and Contract Changes

Before parallel implementation, the team records the endpoint, JSON field names, required types, status codes and at least one representative test response. Freeze this contract before new UI and backend work. Existing code is the *current contract*; proposed changes must be versioned explicitly.

4.2 Existing Diagnose Request

```
POST /api/maintenance/diagnose
Content-Type: application/json

{
  "asset_id": "<existing-asset-id>",
  "symptom_text": "<actual-symptom-description>"
}
```

Do not hardcode a tail number in business logic. A real selected asset from `/maintenance/assets` supplies the identifier. Some labels in the old demo use `813SK`; the team must verify a matching live asset exists before using it.

4.3 Existing Diagnose Response: Field Expectations

```
{
  "asset_id": "<string>",
  "symptom_text": "<string>",
  "ranked_checks": [],
  "evidence": [],
  "rare_cases": [],
  "explanation": "<string>",
  "context_size": {},
  "citation_validation": {
    "valid_ids": [],
    "invalid_ids": [],
    "all_valid": true
  },
  "warnings": []
}
```

This is the *existing field-level contract*, not a promise of populated arrays or a complete type specification. Umar must inspect actual keys of `ranked_checks` and the graph repository's live response. Shahmeer must make the UI tolerant of absent optional fields and explicitly render error states. Never alter a response name unilaterally.

4.4 Other Existing Endpoints

Table 4.1: Minimum Integration Endpoints

Endpoint	Usage / owner
`GET/api/maintenance/assets`	Asset picker / Usman backend; Shahmeer UI.
`GET/api/maintenance/assets/{asset_id}/history`	History timeline / Usman backend; Shahmeer UI.
`GET/api/maintenance/records/{record_id}`	Citation detail / Usman backend; Shahmeer UI.
`POST/api/maintenance/diagnose`	Ranked diagnosis / Umar backend; Shahmeer UI.
`POST/api/maintenance/chat`	Existing maintenance chat / Umar backend; optional for Mid-I.
`GET/api/maintenance/review-queue`	Review queue / Usman backend; Shahmeer UI.
`POST/api/maintenance/review-queue/{record_id}/accept`	Accept review / Usman backend.
`POST/api/maintenance/review-queue/{record_id}/reject`	Reject review / Usman backend.
`GET/api/health`	Runtime readiness / Usman.

4.5 Error Response Agreement

- **400/422**: missing or invalid input; UI shows validation message.
- **404**: asset/record absent; UI offers asset reselection.
- **503**: model/database required for that operation unavailable; UI does *not* substitute a made-up diagnosis.
- **429/5xx**: external rate limit/backend failure; UI displays controlled retry information.
- **200 with empty evidence**: do not imply conclusive diagnosis; show insufficient-evidence state.

These are desired error-display behaviors. Usman and Umar should keep existing HTTP behavior unless a shared contract change is approved and regression tested.

4.6 Cross-Agent Integration Handshake

Umar delivers to Shahmeer: endpoint sample request, actual response fixture, nested `ranked_checks` schema, error examples and known LLM prerequisites. Usman delivers: asset/history/review fixtures and verified service health fields. Shahmeer confirms UI integration using both fixtures and a live call. Each handshake is attached to the relevant PR.

5 Shahmeer Atif: Frontend and Diagnostic Experience

5.1 Domain and Exclusions

Ownership: maintenance-only React UX, asset browsing, diagnostic session layout, evidence presentation, route/navigation cleanup, Axios adapters, frontend tests and end-to-end manual walkthrough. No graph algorithm, LLM API or backend edits.

5.2 Work Package S1: POC Preservation and Cleanup

Inspect: `frontend/src/App.jsx`, `frontend/src/components/Sidebar.jsx`, `frontend/src/context/DomainContext.jsx`, and maintenance pages.

Tasks:

☐ Document current routes and identify software-domain routes to hide from navigation.

☐ Preserve working maintenance URLs (/app/assets, /app/diagnostics, /app/review-queue, etc.).

☐ Remove the confusing domain-switcher path from the Mid-I experience without breaking maintenance context.

☐ Make the main landing action open the maintenance workspace; test browser refresh and deep links.

☐ Ensure legacy software-only screens are not presented as proposed Halite maintenance features.

Acceptance: Navigation contains only intentional maintenance destinations; frontend builds; maintenance deep links work.

5.3 Work Package S2: Asset List and History

Edit: `frontend/src/pages/Assets.jsx`, `frontend/src/pages/AssetHistory.jsx`, and relevant components.

☐ Bind asset selection to `maintenanceAPI.assets()`, not static options.

☐ Show asset identifier, report count and useful metadata if returned.

☐ On selection, open history for the exact backend asset ID; never infer it from the UI label.

☐ Sort/display the timestamps and record IDs returned by the backend; display missing dates honestly.

☐ Make a source record clickable and route it to the existing record detail interface.

☐ Display loading, empty corpus, 404 and network-failure states.

Acceptance: At least one real asset's history can be inspected; a known record ID matches the backend response.

5.4 Work Package S3: Diagnostic Workspace

Edit: `frontend/src/pages/Diagnostics.jsx`, plus self-owned components as needed.

☐ Display a persistent selected-asset context for the current page.

☐ Provide symptom text input with clear button and validation.

☐ Invoke the existing `maintenanceAPI.diagnose(assetId,symptomText)` helper.

☐ Show a visible progress indicator during the request and prevent accidental duplicate submissions.

☐ Render returned `ranked_checks` using actual verified field names.

☐ Render the explanation separately from ranked checks; do not invent details or confidence levels.

☐ Render `warnings` and citation-validation status transparently.

☐ Show specific empty/no-history/model-unavailable states; do not hide backend errors.

Acceptance: The UI performs a live end-to-end diagnostic request. A failed backend shows a recoverable message, not a fake success state.

5.5 Work Package S4: Evidence Navigation and Review UI

Edit: `frontend/src/pages/ServiceRecords.jsx`, `frontend/src/pages/ReviewQueue.jsx`, and front-owned components.

☐ Make returned evidence record IDs open the record view.

☐ Display source metadata available from API (date, component, status).

☐ Distinguish superseded, pending-review and other states only when sourced from backend.

☐ Make existing review queue readable and non-misleading; accept/reject state follows server confirmation.

☐ Show helpful guidance for empty review queue.

Acceptance: At least one diagnosis-to-evidence click and one review queue navigation action work without console errors.

5.6 Work Package S5: Frontend Quality and Tests

☐ Extend `frontend/src/test/Diagnostics.test.jsx` for loading/success/503/empty-evidence states.

☐ Extend Assets and ReviewQueue tests using mocked Axios responses.

☐ Run `npmtest` and `npmrunbuild`; record command results.

☐ Check overflow, clickable targets, broken imports and navigation refresh behavior.

☐ Capture screenshots for documented demonstration states.

Hand-off files: UI screenshots, frontend test logs, exact routes, API integration notes, frontend PR link and a short demo script.

5.7 Shahmeer: Strict Do-Not-Touch List

`backend/api/`, `backend/graph/`, `backend/services/`, `backend/main.py`, `docker-compose.yml`, `.github/workflows/`. If an API field is missing, request the owner to modify the server and use a temporary local fixture in a test, not an unapproved backend rewrite.

6 Muhammad Umar: Graph, Retrieval and Ranking

6.1 Domain and Exclusions

Ownership: Neo4j schema and graph queries; graph-backed diagnostic context; `/diagnose` and `/chat` route internals; candidate ranking; citation verification in diagnostic responses; diagnostic backend tests. Do not edit the React code, ingest/review API, Docker/CI or application router registration.

6.2 Work Package U1: Audit Existing Diagnostic Implementation

Inspect: `backend/services/diagnostic_retrieval.py`, `backend/api/routes/diagnostics.py`, `backend/graph/queries.py`, tests and demo documentation.

☐ Write a short inventory of currently executed functions and their inputs/outputs.

☐ Identify the real graph query path for asset history and fleet matching.

☐ Record whether ranking currently uses word overlap, JASC/part matching, frequency/lift or any actual embedding index.

☐ Explicitly label methods that are not yet hybrid semantic-vector retrieval.

☐ Confirm the schema and data required by `ranked_checks`.

☐ Verify no POC reported result is being mislabeled as diagnostic accuracy.

Acceptance: One-page technical state note with executed code references and actual test results.

6.3 Work Package U2: Real-Data Graph Retrieval

☐ Test history queries with an actual corpus-loaded Neo4j instance, not only the in-memory demo.

☐ Ensure asset-specific records are retrieved with record ID, date, component, text and status as available.

☐ Preserve existing recorded chronological relations; avoid fabricating graph edges.

☐ Use bounded record limits so queries remain responsive on dense assets.

☐ Confirm similarly described symptoms return source-backed records rather than model-created facts.

☐ Support the no-history scenario without uncontrolled exceptions.

Acceptance: A reproducible query returns existing graph records and source identifiers; unknown asset returns controlled behavior.

6.4 Work Package U3: Deterministic Initial Fault Ranking

☐ Preserve the working current ranker unless a specific failing test requires change.

☐ Document scoring factors: symptom match, part/JASC context, same-asset versus fleet evidence, source state.

☐ Make ordering deterministic for equal scores (stable tie-breaking).

☐ Show explicit evidence for every ranked candidate.

☐ Mark superseded historical attempts rather than deleting them or treating them as verified causes.

☐ Do not call numerical ranking scores calibrated fault probabilities.

Acceptance: Repeated identical requests against fixed fixtures give the same candidate order with provenance.

6.5 Work Package U4: Citation Validation and LLM Boundary

☐ Inspect existing `_validate_citations` and its regex assumptions; test real record identifiers.

☐ Prevent fabricated IDs being presented as confirmed evidence.

☐ Keep a visible diagnostic error if live Groq is unavailable and no valid cache exists.

☐ Use LLM only to explain structured evidence; do not invent a procedure or repair approval.

☐ Do not silently weaken validation to make a presentation look better.

☐ Coordinate any new response field with Shahmeer *before* changing its name.

Acceptance: A false record ID is rejected/flagged; a real ID resolves; a missing live-model dependency produces an explained failure.

6.6 Work Package U5: Diagnostic Backend Tests

☐ Run existing `test_diagnostic_retrieval.py` and relevant diagnostic route tests; capture baseline failures.

☐ Add positive case: matching history and deterministic ranked order.

☐ Add negative case: unknown asset or no relevant history.

☐ Add security/correctness case: invalid citation/reference.

☐ Add dependency case: model unavailable.

☐ Document which tests use mocks and which use real Neo4j.

Hand-off files: diagnostic API fixture, response schema note, backend test logs, graph query explanation, benchmark caveat and PR link.

6.7 Umar: Explicitly Deferred Until After Mid-I

Training a new model, full vector-index architecture if not already working, statistically validated fault-probability calibration, broad component dependency graphs and advanced multi-turn session persistence. Stabilize *what currently exists* first.

7 Muhammad Usman Al Haq: Data, Conflict and Delivery

7.1 Domain and Exclusions

Ownership: maintenance asset/record/review routes; data ingestion; conflict engine; backend app wiring; environmental configuration; test infrastructure; GitHub Actions; Compose; deployment instructions; operations evidence. No unilateral changes to the diagnostic route or graph schema; no React page edits.

7.2 Work Package A1: Data and Environment Baseline

☐ Read `README.md`, `docs/maintenance_handoff.md`, `docs/maintenance_state_report.md`, `docker-compose.yml` and environment example.

☐ Verify graph can start on Docker and `DEMO_MODE=false` connects to the intended instance.

☐ Run `GET/api/health` and record graph mode, corpus loaded/count and model mode.

☐ Identify which data import command populates the maintenance corpus without unintended duplicate inserts.

☐ Select a documented real asset and conflict case for deterministic demo use.

☐ Verify credentials are excluded from Git history and do not appear in evidence screenshots.

Acceptance: startup instructions reproduce graph, backend and meaningful maintenance records. No secrets embedded in code or commits.

7.3 Work Package A2: Maintenance APIs and Conflict Workflow

☐ Verify existing asset list, asset history and record detail endpoints.

☐ Run `test_api_maintenance.py`, `test_conflict_engine.py` and `test_maintenance_agent.py`.

☐ Confirm review queue returns documented classifications and source record references.

☐ Confirm accept/reject requests do not destroy original records or incorrectly claim adjudicated diagnoses.

☐ Test no-record, duplicate and invalid-input scenarios.

☐ Coordinate source/asset field expectations with Shahmeer and Umar.

Acceptance: at least one real review case displays; accept/reject semantics are testable using safe seeded/test data, not by damaging the demo corpus.

7.4 Work Package A3: GitHub Actions and Repository Health

☐ Create `.github/workflows/ci.yml` and a meaningful workflow on pull request/push.

☐ Set up Python dependency install and PyTest execution from `backend/`.

☐ Set up frontend `npmci`, `npmtest`, and `npmrunbuild` in CI.

☐ Ensure workflow fails when a test or build fails; do not mask exit codes.

☐ Configure safe test environment; do not upload real `.env` to GitHub.

☐ Capture a passing GitHub Actions run URL and PR evidence.

☐ Arrange controlled staging deployment with health/smoke verification; explain limitations if automated CD cannot be completed.

Acceptance: one successful real hosted workflow after team integration. A YAML file alone does not satisfy the CI requirement.

7.5 Work Package A4: Backend Wiring and Documentation

☐ Own backend router imports/registration in `backend/main.py`; do not rename diagnostic endpoints without Umar approval.

☐ Update outdated README quick-start instructions to emphasize maintenance workflow and distinguish real/demonstration modes.

☐ Write `docs/mid1_setup.md`: requirements, environment-variable names, Docker startup, ingestion, backend/frontend run and health verification.

☐ Write `docs/mid1_test_evidence.md`: exact commands, pass/fail totals, CI URL, known limitations.

☐ Record deployment diagram and runtime dependencies for the university report.

Acceptance: another member can reproduce the app from clean checkout using safe sample configuration.

7.6 Usman: Special Data and Threshold Caveats

Prior documentation describes the existing conflict engine as conservative and includes threshold-tuning concerns on a small test subset. Do not change `MAINT_CONFIDENCE_THRESHOLD` solely to produce auto-accepted results in the demo. If experimenting, record threshold, sample size and independent evaluation limitations. Preserve the historical `GO` negative result: the data does not validate broad component-dependency edge mining.

7.7 Usman: Deliverable Handoff

Runtime checklist, redacted environment configuration, startup and data import instructions, conflict/API test logs, GitHub Actions run, deployment URL if available, release tag, known failures and PR link.

8 Day-by-Day Integration Protocol

8.1 Day 0: Contract Lock (Immediately)

1. All three pull the same baseline commit and report SHA, current branch and working tree state.
2. Usman confirms graph/data/runtime; Umar calls diagnosis API; Shahmeer inspects live UI.
3. Agree response structures; add `docs/mid1_contract.md` through a single document owner.
4. Assign feature branches and exact file ownership; freeze route renames.
5. Write one 8–12 step demo scenario with a *verified real* asset and record IDs.

8.2 Day 1: Parallel Development

Table 8.1: Immediate Parallel Work

Member	Implementation	Acceptance evidence
Shahmeer	Navigation, assets, diagnostics/error states, evidence click-through.	Screenshots, Vitest results, successful frontend build.
Umar	Verified graph/context retrieval, ranker stability, citations, diagnostic tests.	API JSON fixture, graph query proof, PyTest results.
Usman	Dataset/health, asset/review routes, GitHub workflow, runbook, deployment.	Health payload, real review case, CI run, setup log.

8.3 Day 2: Integrate Through Public Interfaces

1. Usman merges the stable backend data/operations PR first if it modifies app setup.
2. Umar merges diagnostic PR after rebasing onto the integration branch and running its focused tests.
3. Shahmeer merges the UI PR after using actual integrated JSON responses.
4. Run all backend/frontend tests and full app build, then perform the complete live scenario.
5. Fix only release-blocking regressions; each owner fixes their own files.

This is a suggested ordering. If there are no cross-file dependencies, independent PRs may merge in another order after green CI.

8.4 Day 3: Release and Rehearsal

☐ Freeze a release candidate and record commit SHA.

☐ Execute the positive diagnostic case, correction case and one failure case.

☐ Check `/api/health` is honest about runtime mode and corpus.

☐ Capture hosted CI run and deployment evidence.

☐ Produce screenshots, code ownership record and report appendices.

☐ Rehearse questions about the algorithms and the provenance of every shown source.

8.5 Late Deadline Condition

Any work on 17–19 October is **post-deadline polish unless an authorized later evaluation date is explicitly confirmed**. Do not schedule mandatory passing features after the 16 October announced window.

9 Git and AI Agent Collaboration Rules

9.1 Safe Branch Routine

```
git status
git switch main
git pull --ff-only origin main
git switch -c feat/mid1/<assigned-area>
# Make ONLY the files assigned to this member.
# Run focused tests; inspect git diff before committing.
git add <only-owned-paths>
git commit -m "mid1: <specific change>"
git push -u origin feat/mid1/<assigned-area>
# Open a reviewed pull request. Do not force-push shared branches.
```

If the team uses a `develop` integration branch, base feature branches on that branch and merge PRs into it; do not mix both policies. Only the release coordinator fast-forwards a verified commit into `main`.

9.2 What Every Agent Must Do Before Coding

1. Inspect the actual code and tests in its ownership scope.
2. Present a short plan listing exact files to modify and endpoint contracts touched.
3. Run the existing relevant tests and note baseline failures.
4. Check whether the desired functionality is already implemented.
5. Identify dependencies owned by another member; request changes instead of editing their files.
6. Make the smallest viable change; preserve backwards compatibility whenever possible.
7. Write tests first or alongside each change; run them before asking for review.

9.3 What Every Agent Must Never Do

- Never rewrite the project, change framework or create a separate project root.
- Never edit `.env` secrets into tracked files or print secret values.
- Never run destructive Neo4j wipes or reseed the only demo dataset without backup and user approval.
- Never delete tests or silence CI failures to obtain a green badge.
- Never change public API fields without a cross-team contract note and updated tests.
- Never switch to a mock graph while claiming live real-data success.
- Never force-push `main`/`develop` or merge unreviewed code.
- Never infer implementation success from AI-generated code; execute commands.

9.4 PR Template (Use as Written)

```
TITLE: mid1: <feature or fix>
OWNER: Shahmeer / Umar / Usman
SCOPE: <one area only>
FILES CHANGED: <paths>
CONTRACT CHANGE: none / linked approved decision
TEST COMMANDS: <commands actually executed>
RESULTS: <pass/fail counts, failures if any>
DEMO EVIDENCE: <screenshot, JSON, workflow URL>
DEPENDENCIES: <other PRs>
KNOWN LIMITATIONS: <factual limitations>
REVIEWER: <member not author>
```

9.5 Conflict Resolution Policy

If two PRs touch the same file, pause one and have its owner manually rebase and reconcile; do not allow agents to choose arbitrary conflict-marker branches. Run both feature tests after resolution. Shared conflict resolutions require a second human reviewer.

10 University Mid-I Deliverables and Evidence Map

10.1 Rubric-Aligned Evidence

The supplied handbook's Mid-I table calls for project vision, existing-product research, use cases, SSD/SRS, test plan, software development plan, wireframes/screens, ethical/legal/DEI discussion, appropriate structure/behavior/data designs, algorithm design, implementation evidence, static checks, unit/system tests, CI/CD, deployment and setup instructions, and a working code demo. The supplied current assessment form also emphasizes a live core module, owner evidence and meaningful tests.

Table 10.1: Required Evidence Production

Artifact / evidence	**Lead**	**Evidence to provide**
Vision, problem and scope	Shahmeer	Maintenance-only scope, problem statement, modules, clear exclusions.
R&D and competitor comparison	Umar	Related approaches, research gap, baselines, clear CCP framing.
Use case diagram, SSD, SRS	Shahmeer	UC asset lookup/diagnosis/evidence/review; FR/NFR with testable criteria.
Domain model, components, layers	Umar	Executed architecture and correct graph/runtime boundary.
Data/schema/ER or graph representation	Umar	Actual Existing graph schema and source IDs.
Behavior/sequence/DFD	Shahmeer	Request to retrieval to ranking to UI, including error branches.
Algorithm and pseudocode	Umar	Candidate retrieval/ranking/citation logic matching implementation.
Implementation technical detail	All	Real files, functions, commit SHA, tests and screenshots per owner.
Test plan, test cases, results	Usman	Unit, integration, negative case, actual pass/fail outputs.
CI/CD branch/PR evidence	Usman	Hosted Actions logs, owners, reviewed PRs, release tag.
Deployment/setup/tool manual	Usman	Reproducible steps, environment names, health verification.
Ethics, privacy, AI disclosure	Shahmeer with all	Actual external AI usage, data/privacy and safety limits.

Artifact / evidence	Lead	Evidence to provide
Iteration status/action register	Shahmeer with all	Planned vs actual, justified deviations, feedback and corrections.
Working demonstration	All	One connected scenario, explanation by each technical owner.

10.2 Report Assembly Guidance

The supplied Mid-I report template's chapters include introduction; research; proposal feedback/iteration plan; requirements; design; iteration implementation; testing and early validation; project management evidence; remaining work; conclusion; and appendices. This execution blueprint *does not replace that official report.* Each owner supplies verified facts to those chapters. Do not place this entire internal agent handoff document into the submission as if it were the implementation report.

10.3 Evidence Folder Layout (New, Proposed)

```
docs/mid1/
  00_status_and_release.md
  01_approved_contracts.md
  02_frontend_evidence.md
  03_graph_diagnostic_evidence.md
  04_data_conflict_devops_evidence.md
  05_tests_and_ci.md
  06_demo_script.md
  07_risks_and_disclosures.md
  screenshots/        # redacted, real runtime only
  fixtures/           # small sanitized sample JSON only
```

Coordinator ownership: Usman creates the directory scaffold and runtime/CI files; Shahmeer writes frontend/demo files; Umar writes diagnostic/graph files. Each document has a single editor.

11 Test Cases, Failure Cases and Acceptance Matrix

Table 11.1: Mid-I End-to-End Acceptance Tests

ID	Scenario	Owner	Expected result
T01	Frontend app opens maintenance workspace.	Shahmeer	No broken route or fatal console errors.
T02	API lists loaded maintenance assets.	Usman	Real identifiers, expected response shape.
T03	Selected asset history is displayed.	Shahmeer	Dated real records with IDs.
T04	Diagnosis request returns evidence.	Umar	Actual graph-backed result and ranking.
T05	Explanation cites a real record.	Umar	ID verified, source opens.
T06	Unknown asset request.	Umar	Controlled failure; no invented history.
T07	LLM is unavailable.	Umar + Shahmeer	Honest 503/recoverable UI or explicitly labeled cached result.
T08	Empty/ambiguous evidence.	Umar + Shahmeer	Uncertainty or clarification, not certainty.
T09	Review queue shows known case.	Usman	Real recorded classification.
T10	Review action on test case.	Usman	Persisted state and safe audit behavior.
T11	Frontend automated tests.	Shahmeer	Passing Vitest run/build.
T12	Backend automated tests.	Umar + Usman	Passing focused PyTest suite.
T13	Pull-request CI workflow.	Usman	Hosted GitHub checks actually run and pass.
T14	Real graph health.	Usman	`graph_mode=neo4j`; corpus present.
T15	Deployment/setup on clean machine.	Usman	Documented reproducible startup.
T16	Legacy screen isolation.	Shahmeer	Maintenance nav does not expose dropped domain.

11.1 No-Mocking Rule for the Evaluator Demo

Mocks are acceptable in unit tests and front-end component tests. In the **live integrated demo**, the main asset/history/diagnosis/conflict path must be backed by actual running services and records. A prerecorded video is an explicit backup, not the primary proof of implementation.

11.2 Post-Integration Smoke Commands

Run these from the actual project root, with safe local environment configured:

```
docker compose up -d neo4j
# In a separate terminal:
cd backend
python -m pip install -r requirements.txt
uvicorn main:app --reload --port 8000
# In another terminal:
cd frontend
npm ci
npm run dev
# Checks, from correct project directories:
python -m pytest -q
npm test
npm run build
```

Qualification: Commands are source README-based conventions, not verified to pass on this system or all operating systems. The team must run them, update platform-specific instructions and document missing environment variables. A live diagnosis also requires a functioning configured model or valid explicitly identified cached result.

12 Live Mid-I Demonstration Script and Defense Preparation

12.1 Demo Order (Suggested 8–12 Minutes)

1. **Shahmeer:** Introduce Halite as maintenance diagnostic support; navigate to Assets and show the real data-driven UI (about 1 minute).
2. **Usman:** Show health fields, actual Neo4j runtime, corpus count and source provenance (about 1 minute).
3. **Shahmeer:** Open a real asset and navigate its chronological history (about 1 minute).
4. **Umar:** Explain selected symptom, candidate retrieval and deterministic scoring (about 1 minute).
5. **Shahmeer:** Trigger live diagnostic query, show ranked checks, cited evidence and status (about 2 minutes).
6. **Usman:** Show a real conflict/review case and safe failure policy (about 1 minute).
7. **All:** Show a failing/empty case and explain why the system refuses to invent records (about 1 minute).
8. **Usman:** Show GitHub branch/PR activity and successful Actions run (about 1 minute).
9. **All:** Explain contribution split and remaining work; take questions.

Times are suggested; adjust to the actual allotted evaluation duration.

12.2 Questions Shahmeer Must Answer

- How do the Assets and Diagnostics pages obtain real data?
- What is the API contract, and how are 404/503/empty states handled?
- How does a user open and verify an evidence citation?
- How were legacy software-domain screens isolated without breaking the application?
- How do frontend tests verify the live API integration behavior?

12.3 Questions Umar Must Answer

- Why use Neo4j rather than only keyword search?
- Which current graph edges are documented and which causal edges are *not* justified?
- Which ranker features are actually implemented, and how do they differ from a trained predictive model?
- What do the record citations prove and what do they not prove?
- How will diagnostics be evaluated separately from the previous conflict-detection F1?

12.4 Questions Usman Must Answer

- How were FAA maintenance records ingested and identified?
- What do `graph_mode`, `llm_mode` and corpus health mean?
- What are the conflict classes and why do uncertain cases require review?
- What exactly runs in GitHub Actions and what prevents a broken merge?
- How can another evaluator reproduce the working application safely?

12.5 Adversarial Demo Cases

1. User enters a symptom for an asset with no history: system does not invent a repair.
2. A cited record was superseded: system displays its historical status.
3. Live LLM is unavailable: request fails transparently or clearly identifies an appropriate cache path.
4. A model mentions a fabricated source: validator flags/rejects the source.
5. GitHub CI contains a broken test: pipeline should fail, not silently pass.

13 Risk Register and Release Freeze

Table 13.1: Immediate Delivery Risks

ID	Risk	Mitigation / owner
R1	Too much new code in one day	Keep current stable routes, fix only blockers; all.
R2	Graph unavailable / demo fallback	Usman verifies Neo4j and health; Umar verifies queries.
R3	Live Groq key/rate unavailable	Umar verifies dependency and proper 503; Shahmeer error UI.
R4	Different API JSON assumptions	Contract freeze and real response fixture; all.
R5	Merge conflicts from AI agents	One owner per file, small PRs, no unapproved path changes.
R6	Untracked or exposed secrets	Usman audits Git and rotates compromised tokens.
R7	Wrong dataset/version	Usman records runtime count, asset/record IDs and fixture version.
R8	No hosted CI proof	Usman publishes working Actions run before release.
R9	Old software scope leaks into UI	Shahmeer isolates navigation, tests deep links.
R10	Misstated diagnostics performance	Umar separates POC conflict metrics from diagnostic evidence.
R11	Missing university artifacts	Each owner supplies screenshots/tests/design; Shahmeer assembles.
R12	Evaluation date confusion	Confirm with supervisor, but prepare for 16 October.

13.1 Release Candidate Check

☐ Known baseline commit and three owner PRs recorded.

☐ Source modified only through agreed ownership boundaries.

☐ Backend imports successfully and real graph queries run.

☐ Frontend build succeeds and critical screens render.

☐ Backend/frontend test reports captured.

☐ Hosted GitHub Actions run captured.

☐ Demo asset/correction record IDs verified.

☐ Positive, negative and failure demos rehearsed.

☐ Report appendices and owner evidence complete.

☐ No credentials/screenshots leaked.

☐ Release tag, runtime settings (redacted), and limitations recorded.

14 Ready-to-Paste Instructions for Coding Agents

14.1 Global Instructions (Give to All Three Agents)

The following text may be copied into each coding-agent session. Replace placeholders with the actual repository and branch. Require the agent to ask before editing outside its allowed scope.

```
You are working on Halite FYP-I Mid, a MAINTENANCE-ONLY
AI diagnostic support app. This is an EXISTING React/Vite +
FastAPI + Neo4j repository, not a blank project.
Read the FYP-I Mid execution blueprint and inspect actual files.
Preserve functioning code. Do not touch the dropped software
product domain except for specifically owned isolation work.
Do not invent test outcomes, credentials, source record IDs,
model accuracy or completed features. Avoid destructive DB actions.
Before changes: show baseline SHA, branch, file inventory,
current test status, exact intended files and contract impacts.
Edit ONLY your assigned paths; request cross-owner changes.
Use a focused feature branch and small commits. Run relevant
tests. Provide file list, summary, commands and actual results.
Never push secrets or commit .env. Never merge to main yourself.
```

14.2 Shahmeer Agent Prompt

```
ROLE: Shahmeer Atif -- sole frontend owner.
BRANCH: feat/mid1/frontend-diagnostics
ALLOWED: frontend/src/** and frontend tests.
OBJECTIVE: Enhance existing Assets, AssetHistory, Diagnostics,
ServiceRecords and ReviewQueue; show real record citations and
clear loading/empty/error states. Hide obsolete software-domain
navigation without breaking maintenance URLs. Reuse existing
maintenanceAPI methods in frontend/src/api/client.js.
DO NOT EDIT backend/**, Docker, CI YAML or .env.
CONTRACT: consult Umar for /maintenance/diagnose response; Usman
for assets/history/records/review. Request API fixes by issue.
VERIFY: npm test; npm run build; live diagnosis-to-source click.
DELIVER: PR, screenshots, fixtures, test results and UI demo steps.
```

14.3 Umar Agent Prompt

```
ROLE: Muhammad Umar -- graph + diagnostic algorithm owner.
BRANCH: feat/mid1/graph-diagnostics
ALLOWED: backend/api/routes/diagnostics.py;
backend/services/diagnostic_retrieval.py; backend/graph/**;
backend/tests/test_diagnostic_retrieval.py and diagnostic tests.
OBJECTIVE: Audit and stabilize REAL Neo4j asset retrieval,
ranked_checks, evidence IDs, candidate ranking and the current
citation validator. Document actual scoring, no-history and
LLM-unavailable behavior. Preserve existing request/response keys.
DO NOT EDIT frontend/**, backend/main.py, conflict engine,
maintenance ingestion routes, Docker or CI YAML.
VERIFY: focused PyTest and one real-graph query with correct IDs.
DELIVER: response fixture, scoring explanation, graph evidence,
positive/failure tests and PR.
```

14.4 Usman Agent Prompt

```
ROLE: Muhammad Usman Al Haq -- maintenance data / conflict /
backend wiring / CI-CD / deployment owner.
BRANCH: feat/mid1/maintenance-devops
ALLOWED: backend/api/routes/maintenance.py;
backend/services/conflict_engine.py;
backend/agents/maintenance_agent.py; backend/main.py;
backend/tests/test_api_maintenance.py and conflict tests;
.github/workflows/**; docker-compose.yml; README and setup docs.
OBJECTIVE: Prove real corpus health and asset/review API behavior,
add hosted GitHub Actions to run actual PyTest + npm tests/build,
create reproducible setup and deployment/health evidence.
DO NOT EDIT frontend/** or Umar's diagnostic/graph files.
Never commit actual .env secrets. Never make an evaluation threshold
change solely to improve demo optics.
VERIFY: backend tests, GitHub Actions URL, graph_mode=neo4j,
maintenance_corpus_loaded=true and reproducible startup.
DELIVER: PR, CI logs, setup manual, review evidence, health JSON.
```

14.5 Agent Stop-and-Ask Conditions

Every agent must pause and notify its human owner if: (1) another member's file must change; (2) an existing endpoint requires an incompatible response change; (3) the demo dataset may be overwritten; (4) keys or credentials appear in history; (5) tests fail for unrelated pre-existing reasons; (6) a major library/framework migration appears necessary; or (7) the verified scope conflicts with a report statement.

A Mid-I Team Final Checklist

A.1 Shahmeer

☐ Maintenance-only navigation and route smoke test.

☐ Real asset list and asset history work.

☐ Live symptom query shows ranked results.

☐ Evidence citation opens source record.

☐ Empty/error/loading states shown.

☐ Frontend tests and build pass; PR documented.

A.2 Umar

☐ Neo4j diagnostic query returns real source records.

☐ Ranking method and weights actually documented.

☐ Diagnostic response fields frozen and shared.

☐ Citation validator tested with valid and invalid IDs.

☐ No-history and model-unavailable tests captured.

☐ Diagnostic test logs, examples and PR documented.

A.3 Usman

☐ Real data corpus verified, IDs recorded, secrets protected.

☐ Asset/history/record/review endpoints tested.

☐ Historical conflict case works.

☐ GitHub workflow runs real tests and build.

☐ Deployment/setup guide reproduces app.

☐ Health shows real graph and recorded corpus.

☐ Release tag, PR, deployment evidence documented.

A.4 Whole Team

☐ Single end-to-end positive demo works.

☐ Negative/failure demo works without fabricated evidence.

☐ All planned vs achieved statements accurately labeled.

☐ Mid-I report covers handbook-designated artifacts.

☐ Each member can explain their contribution independently.

☐ Evaluation meeting confirmed within authorized dates.

B Evidence Log Template

Field	Value	Example of valid evidence
Baseline SHA	_____	Git commit SHA before sprint.
Release SHA	_____	Integration commit under evaluation.
Frontend PR	_____	Reviewed Shahmeer PR.
Graph PR	_____	Reviewed Umar PR.
DevOps PR	_____	Reviewed Usman PR.
CI run	_____	Hosted Actions URL with green jobs.
Test count	_____	Actual command output, including skipped tests.
Demo asset	_____	Real tail/asset from runtime.
Demo record	_____	Actual source-linked record ID.
Graph mode	_____	Real `/api/health` `graph_mode`.
Model mode	_____	Live/fallback/cached behavior.
Limitations	_____	Explicit missing features/known failures.
Supervisor date	_____	Confirmed appointment and report handoff.

C Source References and Decision Notes

This execution blueprint is based on: (1) *BS-FYP-Handbook-2026_DRAFT.pdf*, specifically iteration 1, the FYP-I Mid deliverable table and FYP-I target guidance; (2) *FYP-1-Mid-Assessment-Guidelines-and-Rubrics.pdf*; (3) *F26_FYP1_Mid_Report_LaTeX_Complete.pdf*, a report structure reference; (4) *Halite_FYP_Report.docx*, project modules, proposed responsibilities and roadmap; (5) *Halite (FYP).zip*, inspected source paths and code declarations; (6) internal maintenance architecture, handoff and state documents contained in that archive; and (7) the two FYP Committee emails supplied in conversation.

Source-derived vs proposed: Existing filenames, declared routes and reported prior evaluation limitations were inspected from source material. The exclusive ownership rules, folder conventions, test gates, branch workflow, schedule breakdown, responses to errors and acceptance matrices are *recommended engineering controls for this sprint*, not pre-existing facts or university-imposed requirements.